

# The Case for a Language Based Variant

- Document number: P0095R0
- Date: 2015-09-24
- Project: ISO/IEC JTC1 SC22 WG21
- Reply-to: David Sankel [david@stellarscience.com](mailto:david@stellarscience.com)

## Introduction

The current library-based variant proposals solve an important need<sup>1</sup>, but are they too complicated for novice users? We show that corner cases force novice users to understand the complexities of SFINE in basic usage, the pitfalls of using types as tags, and the difficulty of writing portable code using a library based variant. All of these problems point to the inclusion of a language-based variant feature in C++.

We propose a simple variant language feature called an enumerated union (enum union) with a basic pattern matching mechanism as a means to solve the aforementioned problems. The design is intended to be relatively easy for compiler implementers to add and flexible for future pattern matching extensions. We argue that an enumerated union complements a library-based variant in the same way that a struct complements a library-based tuple.

The following snippet illustrates our proposed syntax.<sup>2</sup>

```
// This enumerated union implements a value representing the various commands
// available in a hypothetical shooter game.
enum union command {
    std::size_t set_score; // Set the score to the specified value
    std::monotype fire_missile; // Fire a missile
    unsigned fire_laser; // Fire a laser with the specified intensity
    double rotate; // Rotate the ship by the specified degrees.
};

// Output a human readable string corresponding to the specified 'cmd' command
// to the specified 'stream'.
std::ostream& operator<<( std::ostream& stream, const command cmd ) {
    switch( cmd ) {
        case set_score value:
            stream << "Set the score to " << value << ".\n";
        case fire_missile m:
            stream << "Fire a missile.\n";
        case fire_laser intensity:
            stream << "Fire a laser with " << intensity << " intensity.\n";
        case rotate degrees:
            stream << "Rotate by " << degrees << " degrees.\n";
    }
}

// Create a new command 'cmd' that sets the score to '10'.
command cmd = command::set_score( 10 );
```

## Motivation

### The struct/tuple and enumerated-union/variant connection

Basic struct types that have independently varying member variables<sup>3</sup> have a close relationship to the `std::tuple` class. Consider the following two types:

```
// point type as a struct
struct point {
    double x;
    double y;
    double z;
};

// point type as a tuple
using point = std::tuple< double, double, double >;
```

It is clear that both point types above can represent a 3D mathematical point. The difference between these two types is,

essentially, the tag which is used to discriminate between the three elements. In the `struct` case, an identifier is used (x, y, and z), and in the `std::tuple` case, an integer index is used (0, 1, and 2).

Although these two point implementations are more-or-less interchangeable, it is not always preferable to use a `struct` instead of a `std::tuple` nor vice-versa. In particular, we have the following general recommendations:

1. If the type needs to be created on the fly, as in generic code, a `std::tuple` must be used.
2. If an integer index isn't a clear enough identifier, a `struct` should be used.
3. Arguably, if inner types aren't essentially connected or if the structure is used only as the result of a function and is immediately used, a `std::tuple` is preferable.
4. In general, prefer to use a `struct` for improved code clarity.

Some may argue that through use of `std::get`, which allows one to fetch a member of a tuple by type, one can achieve all the benefits of a `struct` by using a tuple instead. To take advantage of this feature, one needs to ensure that each inner type has its own distinct type. This can be accomplished through use of a wrapper. For example:

```
struct x { double value; };
struct y { double value; };
struct z { double value; };

using point = std::tuple< x, y, z >;
```

Now one could use `std::get<x>` to fetch the 'x' value of the tuple, `std::get<y>` for 'y' and so on.

Should we use this approach everywhere and deprecate the use of `struct` in any context? In the author's opinion we should not. The use of wrapper types is much more complicated to both read and understand than a plain `struct`. For example, the wrapper types that were introduced, such as the 'x' type, make little sense outside of their corresponding tuples, yet they are peers to it in scope. Also, the heavy syntax makes it difficult to understand exactly what is intended by this code.

What does all this have to do with enumerated unions? The enumerated union is to `std::variant` as `struct` is to `std::tuple`. A variant type that represents a distance in an x direction, a y direction, *or* a z direction (mathematically called a "copoint") has a similar look and feel to the `std::tuple` version of point.

```
struct x { double value; };
struct y { double value; };
struct z { double value; };

using copoint = std::variant< x, y, z >;
```

This copoint implementation has the same drawbacks that the `std::tuple` implementation of points has. An enumerated union version of copoint, on the other hand, is easier to grok and doesn't require special tag types at all.

```
enum union copoint {
    double x;
    double y;
    double z;
};
```

## SFINE in basic usage

Some variation of the following example is common when illustrating a `std::variant` type:

```
void f( std::variant< double, std::string> v ) {
    if( std::holds_alternative< double >( v ) ) {
        std::cout << "Got a double " << std::get< double >( v ) << std::endl;
    }
    else {
        std::cout << "Got a string " << std::get< std::string >( v ) << std::endl;
    }
};
```

This illustrates how quickly variants can be disassembled when they are simple, but it is hardly representative of how complex variant types are used. The primary problem in the above snippet is that there are no compile-time guarantees that ensure all of the `n` alternatives are covered. For the more general scenario, a `visit` function is provided.<sup>[4](#)</sup>

```
struct f_visitor {
```

```

void operator()( const double d ) {
    std::cout << "Got a double " << d << std::endl;
}
void operator()( const std::string & s ) {
    std::cout << "Got a string " << s << std::endl;
}
};

void f( std::variant< double, std::string > v ) {
    std::visit( f_visitor(), v );
};

```

Aside from the unsightly verbosity of the above code, the mechanism by which this works makes the visitor's `operator()` rules work by SFINE, which is a significant developer complication. Using a template parameter as part of a catch-all clause is going to necessarily produce strange error messages.

```

struct f_visitor {
    template< typename T >
    void operator()( const T & t ) {
        // oops
        std::cout << "I got something " << t.size() << std::endl;
    }
};

void f( std::variant< double, std::string > v ) {
    // Unhelpful error message awaits. Erroneous line won't be pointed out.
    std::visit( f_visitor(), v );
};

```

While the utility of type selection and SFINE for visitors is quite clear for advanced C++ developers, it presents significant hurdles for the beginning or even intermediate developer. This is especially true when it is considered that the `visit` function is the only way to guarantee a compilation error when all cases are not considered.

## Duplicated types: switching on the numeric index

Using types as accessors with a `std::variant` works for many use cases, but not all. If there is a repeated type the only options are to either use wrapper types or to work with the real underlying discriminator, an integer index. To illustrate the problems with using the index, consider the following implementation of `copoint`:

```

using copoint = std::variant< double, double, double >;

```

Use of both `std::get<double>` and the standard `std::visit` are impossible due to the repeated `double` type in the variant. Using the numeric index to work around the issue brings its own problems, however. Consider the following visitor:

```

struct visit_f {
    void operator()( std::integral_constant<std::size_t, 0>, double d ) {
        std::cout << d << " in x" << std::endl;
    };
    void operator()( std::integral_constant<std::size_t, 1>, double d ) {
        std::cout << d << " in y" << std::endl;
    };
    void operator()( std::integral_constant<std::size_t, 2>, double d ) {
        std::cout << d << " in z" << std::endl;
    };
};

```

Here we introduce yet another advanced C++ feature, compile-time integrals. In the opinion of the author, this is unfriendly to novices. The problem of duplicated types can be even more insidious, however...

## Portability problems

Consider the following code:

```

using json_integral = std::variant< int, unsigned, std::size_t, std::ptrdiff_t >;

```

On most platforms, this code will compile and run without a problem. However, if `std::size_t` happens to be `typedef'd` to be the same type as `unsigned` on a particular platform, a compilation error will ensue. The only two options for fixing the error are to

fall back to using the index or to make custom wrapper types.

Also notable is that working with third party libraries that are free to change their underlying types creates abstraction leaks when used with a library-based variant.

```
// Is this code future proof? Not likely. Looks like a foot-gun to me.
using database_handle = std::variant< ORACLE_HANDLE, BERKELEY_HANDLE >;
```

Because enumerated unions require identifiers as tags, they aren't susceptible to this problem:

```
enum union database_handle {
    ORACLE_HANDLE oracle;
    BERKELEY_HANDLE berkeley;
};
```

## Syntax/Semantics Overview

The definition of an enumerated union would have the same syntax as a union, but with an `enum` keyword beforehand as in the following example:

```
// This enumerated union implements a value representing the various commands
// available in a hypothetical shooter game.
enum union command {
    std::size_t set_score; // Set the score to the specified value
    std::monotype fire_missile; // Fire a missile
    unsigned fire_laser; // Fire a laser with the specified intensity
    double rotate; // Rotate the ship by the specified degrees.
};
```

Each member declaration consists of a type followed by its corresponding identifier.

### Construction and Assignment

A enumerated union has a default constructor if its first member also has a default constructor. A default constructed enumerated union is set to the first member's default constructed value.

Assignment at construction can be used to set the enumerated union to a particular value. The enumerated union is used as a namespace when specifying specific alternatives.

```
command cmd = command::set_score( 10 );
```

Enumerated union instances can also be assigned to in the course of a program's execution.

```
cmd = command::fire_missile( );
```

The behavior of assignment in the presence of exceptions is intentionally left unspecified pending conclusions from the current discussions on a library-based variant.

### Switch

The `switch` statement on a variant is modeled after that of an `enum`.

```
switch( cmd ) {
    case set_score value:
        stream << "Set the score to " << value << ".\n";
    case fire_missile m:
        stream << "Fire a missile.\n";
    case fire_laser intensity:
        stream << "Fire a laser with " << intensity << " intensity.\n";
    case rotate degrees:
        stream << "Rotate by " << degrees << " degrees.\n";
}
```

The notable differences are as follows:

1. The `case` clause includes both the alternative identifier and a variable name that is bound to the alternative's value.
2. No `break` statements are necessary since each `case` statement is independent of the others.
3. The variable that is bound to the alternative's value has scope until either the next `case` clause or the end of the `switch` statement.

The `default` keyword can be used as a catch-all, but it does not have a bound variable.

```
// Only handle the 'set_score' command.
switch( cmd ) {
    case set_score value:
        score = value;
    default:
}

```

## Design Considerations

### Would generalized pattern matching solve all the library-based variant problems?

Although generalized pattern matching as proposed at the November 2014 standardization committee meeting<sup>5</sup> would greatly reduce the syntactic overhead of library-based variants, many problems would still remain. In particular, because types are still being used as tags, users will still need to be quite knowledgeable about advanced C++ topics to use them effectively.

### How does the proposed pattern matching compare with generalized pattern matching?

Because there has not been a concrete generalized pattern matching proposal as of yet, it is difficult to make direct comparisons. The design of pattern matching in this proposal has been made *intentionally simple* to ease the burden on compiler implementers and moderate complexity for new users.

On the flip-side, the particular syntax chosen here was also designed to be easily extended to handle more complex pattern matching variations in future revisions of the language. For example:

```
enum union position_command {
    double set_rotation;
    std::pair< double, double > set_position;
};

void f( position_command cmd ) {
    switch( cmd ) {
        case set_rotation r:
            // ...
        case set_position {0.0, 0.0}:
            // handle the origin case specially
            // ...
        case set_position {x, y}:
            // handle other cases
            // ...
    }
}

```

## Conclusion

We conclude that types-as-tags are for astronauts, but variants are for everyone. None of the library implementations thus far proposed are easy enough to be used by beginners; a language feature is necessary. In the author's opinion a library-based variant should complement a language-based variant, but not replace it.

- 
1. Variant: a type-safe union (v4). N4542↩
  2. The `std::monotype` type, which has only one value, is from variant proposal N4542.↩
  3. See [The C++ Core Guidelines](#) rule C.2.↩
  4. Compare that code to the same for an enumerated union:

```
enum union double_or_string {
    double with_double;
    std::string with_string;
};

```

```
void f( double_or_string v ) {  
    switch( v ) {  
        case with_double d:  
            std::cout << "Got a double " << d << std::endl;  
        case with_string s:  
            std::cout << "Got a string " << s << std::endl;  
    }  
}
```

[↩](#)

5. Pattern Matching for C++. Yuriy Solodkyy, Gabriel Dos Reis, and Bjarne Stroustrup.[↩](#)